Studio Nx

S1-08 패턴 API — 함수 설명 합본 (백엔드 211~218)

2026-10-17

목차

1. ai-notes/backend(fastAPI_Python)/S1-08 - 아코디언·피아노롤/211_models_pattern.md
2. ai-notes/backend(fastAPI_Python)/S1-08 - 아코디언·피아노롤/212_models_midi_note.md
3. ai-notes/backend(fastAPI_Python)/S1-08 - 아코디언·피아노롤/213_schemas_pattern.md
4. ai-notes/backend(fastAPI_Python)/S1-08 - 아코디언·피아노롤/214_pattern_service_query.md
5. ai-notes/backend(fastAPI_Python)/S1-08 - 아코디언·피아노롤/215_pattern_service_replace_notes.md
6. ai-notes/backend(fastAPI_Python)/S1-08 - 아코디언·피아노롤/216_pattern_service_update_pattern.md
7. ai-notes/backend(fastAPI_Python)/S1-08 - 아코디언·피아노롤/217_api_patterns_list.md
8. ai-notes/backend(fastAPI_Python)/S1-08 - 아코디언·피아노롤/218_api_patterns_notes.md

Backend(FastAPI/Python) 211 — `Pattern`

항목	내용
파일	`studionx/backend/app/models/pattern.py`
작업	S1-08 아코디언·피아노롤: `patterns` 표 (PRD 02 FR-231, FR-233)
작성	Claude (AI), 2026-10-17, 사용자 검토
이전 · 다음	210 `startSampleEdit` · 212 `MidiNote`

1. 하는 일

MIDI 클립이 가리키는 **노트 묶음(패턴)** 한 건을 담는 표입니다. 노트 자체는 자식 표(`midi_notes`, 212)에 있고, 이 행은 이름과 길이, 스윙만 가집니다.

넣은 것	뜻
`pattern_id`	기본 키 (자동 증가)
`project_id`	어느 프로젝트의 패턴인지 (FK → `projects`, 프로젝트를 지우면 함께 지워짐)
`name`	패턴 이름 (`VARCHAR(100)`)
`color`	클립 색 `#RRGGBB`, 없으면 NULL
`length_tick`	패턴 길이 (1박 = 960 tick, 기본 3840 = 4박)
`swing`	스윙 정도 0.00~1.00 (DB CHECK), 0이면 곧은 박자
`created_at`, `updated_at`	만든 시각·고친 시각

2. 언제 쓰나

쓰는 곳	
`pattern_service.list_patterns` (214)	프로젝트의 패턴 목록을 읽습니다
`pattern_service.get_pattern` (214)	피아노롤이 열 때 이 행과 노트를 함께 읽습니다
`replace_notes` (215)	이 행에 딸린 노트를 통째로 바꿉니다
`update_pattern` (216)	이름·색·길이·스윙을 고칩니다

3. 코드

import (파일 맨 위)

```
from datetime import datetime
from decimal import Decimal
from typing import TYPE_CHECKING

from sqlalchemy import BigInteger, FetchedValue, ForeignKey, Numeric, String, text
from sqlalchemy.dialects.mysql import DATETIME, INTEGER
from sqlalchemy.orm import Mapped, mapped_column, relationship
```

```
from app.db.session import Base

if TYPE_CHECKING:
    from app.models.midi_note import MidiNote
    from app.models.project import Project
```

모델

```
class Pattern(Base):
    __tablename__ = "patterns"

    pattern_id: Mapped[int] = mapped_column(BigInteger, primary_key=True, autoincrement=True)
    project_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("projects.project_id", ondelete="CASCADE")
    )
    name: Mapped[str] = mapped_column(String(100))
    color: Mapped[str | None] = mapped_column(String(7))
    length_tick: Mapped[int] = mapped_column(INTEGER(unsigned=True), server_default=text("3840"))
    swing: Mapped[Decimal] = mapped_column(Numeric(3, 2), server_default=text("0.00"))
    created_at: Mapped[datetime] = mapped_column(
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)")
    )
    updated_at: Mapped[datetime] = mapped_column(
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)"), server_onupdate=FetchedValue()
    )

    project: Mapped["Project"] = relationship(back_populates="patterns")
    notes: Mapped[list["MidiNote"]] = relationship(
        back_populates="pattern",
        order_by="MidiNote.start_tick, MidiNote.pitch",
        cascade="all, delete-orphan",
        passive_deletes=True,
    )
```

문법 풀이

식	뜻
`INTEGER(unsigned=True)`	MySQL 정수(0 이상)입니다. tick 은 음수가 될 수 없습니다
`Numeric(3, 2)`	소수점 둘째 자리까지(0.00~9.99)입니다. `swing` 0.00~1.00 을 담습니다
`server_default=text("3840")`	값을 안 주면 DB 가 채웁니다. 앱에서 안 넣어도 됩니다
`server_onupdate=FetchedValue()`	DB 의 `ON UPDATE` 가 채우는 칸임을 알려 줍니다
`relationship(..., order_by=...)`	노트를 시작 위치, 음높이 차례로 정렬해 읽습니다
`passive_deletes=True`	삭제는 DB 의 `ON DELETE CASCADE` 에 맡기고 앱이 노트를 하나씩 안 지웁니다
`TYPE_CHECKING`	타입 표시용 import 로 실행 중에는 불러오지 않아 순환 참조를 피합니다

4. 왜 이렇게

선택	이유
노트를 자식 표로	한 패턴에 노트가 많아 패턴 행에 담을 수 없습니다. `midi_notes` 가 `pattern_id` 로 가리킵니다

선택	이유
삭제를 DB 에 맡김	프로젝트를 지우면 패턴, 패턴을 지우면 노트가 `CASCADE` 로 함께 지워집니다. 앱이 빠뜨릴 일이 없습니다
tick 단위	편곡·피아노롤이 1박 = 960 tick 을 씁니다. 화면과 DB 가 같은 단위라 변환이 필요 없습니다

5. 시험

`tests/test_pattern_model.py` 에서 표 이름·칸·기본값·관계가 DB(`assert_matches_db`)와 같은지, 노트가 시작 위치·음높이 차례로 정렬되는지 확인했습니다. 6개가 통과합니다.

Backend(FastAPI/Python) 212 — `MidiNote`

항목	내용
파일	`studionx/backend/app/models/midi_note.py`
작업	S1-08 아코디언·피아노롤: `midi_notes` 표 (PRD 02 FR-231, FR-232)
작성	Claude (AI), 2026-10-17, 사용자 검토
이전 · 다음	211 `Pattern` · 213 패턴 스키마

1. 하는 일

피아노롤에서 찍은 노트 한 건을 담습니다. 어느 패턴에 속하는지(`pattern_id`)와 음높이·시작 위치·길이·세기를 가집니다. 겹쳐 찍은 노트(FR-2317)는 같은 음높이·위치로 여러 행이 생깁니다.

칸	뜻
`note_id`	노트 번호 (자동 증가)
`pattern_id`	어느 패턴의 노트인지 (패턴을 지우면 함께 지워짐)
`pitch`	음높이 0~127 (60 = C4)
`velocity`	세기 1~127 (기본 100)
`start_tick`	시작 위치 (1박 = 960 tick)
`length_tick`	길이 (tick)
`channel`	MIDI 채널 0~15 (기본 0)
`release_velocity`	뗄 때 세기 (기본 64)
`is_muted`	소리 끔 (기본 아님)
`probability`	소리 낼 확률 0.00~1.00 (기본 1.00)
`velocity_deviation`	세기 흔들림 (기본 0)

2. 언제 쓰나

쓰는 곳	
`pattern_service._notes` (214)	패턴의 노트를 시작 위치·음높이 차례로 읽습니다
`replace_notes` (215)	지금 노트를 지우고 보낸 목록을 새로 넣습니다
피아노롤 (sequencer-roll.js)	`start_tick`·`length_tick`·`pitch`·`velocity` 이름을 화면과 그대로 맞춥니다

3. 코드

import (파일 맨 위)

```
from decimal import Decimal
from typing import TYPE_CHECKING
```

```
from sqlalchemy import BigInteger, Boolean, ForeignKey, Numeric, text
from sqlalchemy.dialects.mysql import INTEGER, TINYINT
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.db.session import Base

if TYPE_CHECKING:
    from app.models.pattern import Pattern
```

```
class MidiNote(Base):
    __tablename__ = "midi_notes"

    note_id: Mapped[int] = mapped_column(BigInteger, primary_key=True, autoincrement=True)
    pattern_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("patterns.pattern_id", ondelete="CASCADE")
    )
    pitch: Mapped[int] = mapped_column(TINYINT(unsigned=True))
    velocity: Mapped[int] = mapped_column(TINYINT(unsigned=True), server_default=text("100"))
    start_tick: Mapped[int] = mapped_column(INTEGER(unsigned=True))
    length_tick: Mapped[int] = mapped_column(INTEGER(unsigned=True))
    channel: Mapped[int] = mapped_column(TINYINT(unsigned=True), server_default=text("0"))
    release_velocity: Mapped[int] = mapped_column(TINYINT(unsigned=True), server_default=text("64"))
    is_muted: Mapped[bool] = mapped_column(Boolean, server_default=text("0"))
    probability: Mapped[Decimal] = mapped_column(Numeric(3, 2), server_default=text("1.00"))
    velocity_deviation: Mapped[int] = mapped_column(
        TINYINT(unsigned=True), server_default=text("0")
    )

    pattern: Mapped["Pattern"] = relationship(back_populates="notes")
```

문법 풀이

식	뜻
`TINYINT(unsigned=True)`	0~255 작은 정수입니다. pitch·velocity·channel 이 여기 들어갑니다
`Boolean`	참·거짓입니다. DB 에는 `TINYINT(1)` 로 들어갑니다
`Numeric(3, 2)`	`probability` 0.00~1.00 을 담습니다
`server_default=text("100")`	값을 안 주면 DB 가 채웁니다(예: velocity 100)
`relationship(back_populates="notes")`	211 의 `Pattern.notes` 와 짝을 이룹니다

4. 왜 이렇게

선택	이유
노트를 통째로 갈아 끼움	피아노롤은 한 번의 편집으로 여러 노트가 바뀝니다(겹침·이동). 노트마다 번호를 주고받기보다 목록 전체를 다시 보내는 편이 간단하고 어긋날 일이 없습니다 (215)
`probability`·`velocity_deviation`·`is_muted` 미리 둠	당장은 안 쓰지만 FR-232 의 노트 설정을 담을 칸을 지금 만들어 둡니다
`release_velocity` 기본 64	못 박은 소리를 막기 위한 중간값입니다

5. 시험

`tests/test_pattern_model.py` 에서 `midi_notes` 표가 DB 와 같은지, 관계·기본값을 확인했습니다. 통과합니다.

Backend(FastAPI/Python) 213 — 패턴 스키마 (`NoteInput` · `NoteItem` · `NotesSaveRequest` · `PatternUpdateRequest` · `PatternResponse` · `PatternListResponse`)

항목	내용
파일	`studionx/backend/app/schemas/pattern.py`
작업	S1-08 아코디언·피아노롤: 패턴 요청·응답 모양 (PRD 02 FR-231 ~ FR-233)
작성	Claude (AI), 2026-10-17, 사용자 검토
이전 · 다음	212 `MidiNote` · 214 `pattern_service` 조회

1. 하는 일

패턴 API 가 주고받는 JSON 모양을 정합니다. 노트를 **통째로** 주고받으므로(215) 노트 하나의 모양(`NoteInput`)과 여러 개를 담는 요청(`NotesSaveRequest`)을 나눕니다. 칸 이름은 DB(`midi_notes`)와 피아노롤(`sequencer-roll.js`)이 쓰는 이름을 그대로 맞춥니다.

스키마	쓰는 곳
`NoteInput`	노트 한 건(보낼 때). `note_id` 는 받지 않습니다
`NoteItem`	노트 한 건(응답). `NoteInput` 에 `note_id` 를 더한 모양
`NotesSaveRequest`	`PUT /patterns/{id}/notes` — 노트 목록
`PatternUpdateRequest`	`PATCH /patterns/{id}` — 이름·색·길이·스윙
`PatternResponse`	목록·상세·저장 응답이 모두 쓰는 패턴 한 건
`PatternListResponse`	`GET /patterns` — 패턴 목록

2. 언제 쓰나

쓰는 곳	
`api/v1/patterns.py` (217)	요청 본문·응답 모델로 지정합니다
`pattern_service.to_response` (214)	`PatternResponse` 를 만듭니다
`pattern_service.replace_notes` (215)	`NotesSaveRequest.notes` 를 새 행으로 옮깁니다

3. 코드

칸 길이·개수 상한과 타입 별칭

```
PATTERN_NAME_MAX = 100          # patterns.name 칸 길이와 같음
PATTERN_COLOR_MAX = 7           # '#RRGGBB'
NOTES_MAX = 5000                # 실수로 큰 목록을 보내는 것 방지

PatternName = Annotated[str, StringConstraints(strip_whitespace=True, min_length=1, max_length=PATTERN_N
AME_MAX)]
Pitch = Annotated[int, Field(ge=0, le=127)]          # MIDI 0~127
```

```
Velocity = Annotated[int, Field(ge=1, le=127)]        # 1~127 (DB CHECK)
StartTick = Annotated[int, Field(ge=0)]
LengthTick = Annotated[int, Field(ge=1)]
Channel = Annotated[int, Field(ge=0, le=15)]
```

노트 한 건과 목록

```
class NoteInput(BaseModel):
    pitch: Pitch
    start_tick: StartTick
    length_tick: LengthTick
    velocity: Velocity = 100
    channel: Channel = 0
    release_velocity: Annotated[int, Field(ge=0, le=127)] = 64
    is_muted: bool = False
    probability: Decimal = Field(default=Decimal("1.00"), ge=0, le=1, decimal_places=2)
    velocity_deviation: Annotated[int, Field(ge=0, le=127)] = 0

class NoteItem(NoteInput):
    note_id: int

class NotesSaveRequest(BaseModel):
    notes: list[NoteInput] = Field(default_factory=list, max_length=NOTES_MAX)
```

패턴 수정 · 응답

```
class PatternUpdateRequest(BaseModel):
    name: PatternName | None = None
    color: Annotated[str, StringConstraints(strip_whitespace=True, max_length=PATTERN_COLOR_MAX)] | None
= None
    length_tick: Annotated[int, Field(ge=1)] | None = None
    swing: Annotated[Decimal, Field(ge=0, le=1, decimal_places=2)] | None = None

    @field_validator("name")
    @classmethod
    def _name_not_null(cls, value):
        if value is None:
            raise ValueError("name cannot be empty")
        return value

class PatternResponse(BaseModel):
    pattern_id: int
    project_id: int
    name: str
    color: str | None
    length_tick: int
    swing: Decimal
    updated_at: datetime
    notes: list[NoteItem]

class PatternListResponse(BaseModel):
    items: list[PatternResponse]
    total: int
```

문법 풀이

식	뜻
`Annotated[..., Field(ge=…, le=...)]`	값 범위를 붙입니다. 벗어나면 422 로 거부됩니다
`StringConstraints(strip_whitespace=True, ...)`	앞뒤 공백을 먼저 지우고 길이를 잽니다
`Field(max_length=NOTES_MAX)`	목록 길이 상한입니다. 한 패턴 5000개를 넘기면 422 입니다
`Decimal` + `decimal_places=2`	소수 둘째 자리까지 받습니다(`swing`, `probability`)
`Field(default_factory=list)`	요청에 `notes` 가 없으면 빈 목록으로 봅니다
`@field_validator`	값 하나를 더 검사합니다. 보낸 경우에만 돕니다

4. 왜 이렇게

선택	이유
이름을 화면·DB 와 동일하게	`start_tick`·`length_tick`·`pitch`·`velocity`. 피아노롤이 그대로 보내고 그대로 받습니다. 변환 코드가 없습니다
`NOTES_MAX = 5000`	잘못된 화면이 거대한 목록을 보내 서버를 힘들게 하는 것을 막습니다
`note_id` 는 응답에만	노트는 통째 교체라 새 번호가 매겨집니다. 보낼 때 번호를 받아도 쓸 데가 없습니다
`name` 에 null 금지 검사	`PATCH` 는 보낸 칸만 바꾸므로, `name: null` 을 직접 보낸 경우만 막습니다. 안 보내면 검사하지 않습니다

4. 시험

`tests/test_pattern_routes.py` 에서 422(빈 이름·음높이 128·길이 0·채널 16)와 정상 응답 모양을 확인했습니다. 통과합니다.

Backend(FastAPI/Python) 214 — `pattern_service` 조회 (`to_response` · `_note_item` · `_notes` · `_find` · `list_patterns` · `get_pattern`)

항목	내용
파일	`studionx/backend/app/services/pattern_service.py`
작업	S1-08 아코디언·피아노롤: 패턴 목록·상세 (PRD 02 FR-232, FR-233)
작성	Claude (AI), 2026-10-17, 사용자 검토
이전 · 다음	213 패턴 스키마 · 215 `replace_notes`

1. 하는 일

패턴을 찾아 응답 모양으로 바꿉니다. 권한 규칙은 `project_service` 와 같습니다. 조회는 프로젝트 멤버면 누구나, 남의 프로젝트 패턴은 있는지조차 알려 주지 않고 404 로 막습니다(`get_membership`).

함수	하는 일
`_note_item(note)`	`MidiNote` 행 → `NoteItem`
`_notes(db, pattern_id)`	그 패턴의 노트를 시작 위치·음높이 차례로 읽음
`to_response(pattern, notes)`	`Pattern` 행 → `PatternResponse`
`_find(db, user, pattern_id)`	패턴·프로젝트·내 멤버 행을 함께 찾음 (없거나 남의 것이면 404)
`list_patterns(db, user, project_id)`	프로젝트의 패턴 목록 (노트는 빼고 가볍게)
`get_pattern(db, user, pattern_id)`	패턴 한 건과 노트 전부 (피아노롤이 열 때)

2. 언제 쓰나

쓰는 곳	
`api/v1/patterns.py` (217)	목록·상세 라우터가 부릅니다
`replace_notes`·`update_pattern` (215·216)	응답을 만들 때 `to_response` 를 함께 씁니다

```
GET /patterns?project_id=3
└ list_patterns
   ├ get_membership          ← 멤버 아니면 404
   └ SELECT patterns WHERE project_id=3 ORDER BY pattern_id
      └ to_response(notes=[])  ← 목록은 노트를 담지 않는다

GET /patterns/12
└ get_pattern
   ├ _find                   ← 패턴 없거나 남의 것이면 404
   └ _notes(db, 12)          ← 시작 위치·음높이 차례
      └ to_response
```

3. 코드

```
def _note_item(note: MidiNote) -> NoteItem:
    return NoteItem(
        note_id=note.note_id,
        pitch=note.pitch,
        start_tick=note.start_tick,
        length_tick=note.length_tick,
        velocity=note.velocity,
        channel=note.channel,
        release_velocity=note.release_velocity,
        is_muted=note.is_muted,
        probability=note.probability,
        velocity_deviation=note.velocity_deviation,
    )

def _notes(db: Session, pattern_id: int) -> list[MidiNote]:
    return list(
        db.scalars(
            select(MidiNote)
            .where(MidiNote.pattern_id == pattern_id)
            .order_by(MidiNote.start_tick, MidiNote.pitch)
        ).all()
    )

def to_response(pattern: Pattern, notes: list[MidiNote] | None = None) -> PatternResponse:
    rows = pattern.notes if notes is None else notes
    return PatternResponse(
        pattern_id=pattern.pattern_id,
        project_id=pattern.project_id,
        name=pattern.name,
        color=pattern.color,
        length_tick=pattern.length_tick,
        swing=pattern.swing,
        updated_at=pattern.updated_at,
        notes=[_note_item(note) for note in rows],
    )

def _find(db: Session, user: User, pattern_id: int) -> tuple[Pattern, Project, ProjectMember]:
    pattern = db.get(Pattern, pattern_id)
    if pattern is None:
        raise AppError(ErrorCode.NOT_FOUND, 404, "Pattern not found.")
    project, member = project_service.get_membership(db, pattern.project_id, user)
    return pattern, project, member

def list_patterns(db: Session, user: User, project_id: int) -> PatternListResponse:
    project_service.get_membership(db, project_id, user)
    rows = list(db.scalars(
        select(Pattern).where(Pattern.project_id == project_id).order_by(Pattern.pattern_id)
    ).all())
    return PatternListResponse(items=[to_response(p, notes=[]) for p in rows], total=len(rows))

def get_pattern(db: Session, user: User, pattern_id: int) -> PatternResponse:
    pattern, _project, _member = _find(db, user, pattern_id)
    return to_response(pattern, _notes(db, pattern.pattern_id))
```

문법 풀이

식	뜻
`notes: list[MidiNote]	None = None`
`pattern.notes if notes is None else notes`	"안 넘겼다"와 "빈 목록을 넘겼다"를 구분합니다
`db.get(Pattern, id)`	기본키로 한 행을 찾습니다 (없으면 `None`)
`raise AppError(ErrorCode.NOT_FOUND, 404, ...)`	06번 오류 처리기가 같은 모양으로 응답합니다
`select(...).order_by(...)`	SQL 로 정렬해 읽습니다
`notes=[]`	목록 응답은 빈 목록을 넘겨 노트를 빼둡니다

4. 왜 이렇게

선택	이유
목록은 노트를 안 담음	프로젝트에 패턴이 많으면 노트까지 다 실으면 무겁습니다. 목록은 가볍게, 노트가 필요하면 상세로 받습니다
`_notes` 로 직접 조회	통째 교체(215) 직후에는 세션에 옛 노트가 남아 있을 수 있습니다. 관계 대신 쿼리로 읽어 최신 상태를 봅니다
남의 것은 404	있는지조차 알려 주지 않는 편이 안전합니다. `get_membership` 이 이 규칙을 이미 갖고 있습니다
`_find` 가 셋을 함께 돌려줌	뒤 두 함수(215·216)가 권한 검사를 위해 프로젝트·멤버 행을 같이 씁니다

5. 시험

`tests/test_pattern_routes.py` 에서 목록·상세 응답, 비멤버 404 를 확인했습니다. 통과합니다.

Backend(FastAPI/Python) 215 — `replace_notes()`

항목	내용
파일	`studionx/backend/app/services/pattern_service.py`
작업	S1-08 아코디언·피아노롤: 노트 통째 교체 (PRD 02 FR-231)
작성	Claude (AI), 2026-10-17, 사용자 검토
이전 · 다음	214 `pattern_service` 조회 · 216 `update_pattern`

1. 하는 일

피아노롤이 보낸 노트 목록으로 그 패턴의 노트를 **통째로 갈아 끼웁니다**. 지금 붙은 노트를 모두 지우고 보낸 것을 새로 넣습니다. 한 트랜잭션이라 반쪽 상태가 남지 않습니다. 빈 목록이면 노트를 모두 지웁니다. 저장은 OWNER·EDITOR 만 하고, VIEWER 는 403 입니다.

2. 언제 쓰나

쓰는 곳	
`PUT /patterns/{id}/notes` (218)	피아노롤이 노트를 바꿀 때마다 화면의 노트 전부를 다시 보냅니다
피아노롤 (sequencer-roll.js)	노트 찍기·옮기기·지우기·겹쳐 찍기가 끝나면 이 API 를 부릅니다 (연결은 S1-10)

3. 코드

```
def replace_notes(
    db: Session, user: User, pattern_id: int, data: NotesSaveRequest
) -> PatternResponse:
    pattern, _project, member = _find(db, user, pattern_id)
    project_service.require_role(member, MemberRole.OWNER, MemberRole.EDITOR)

    db.execute(delete(MidiNote).where(MidiNote.pattern_id == pattern.pattern_id))
    db.add_all(
        MidiNote(
            pattern_id=pattern.pattern_id,
            pitch=note.pitch,
            start_tick=note.start_tick,
            length_tick=note.length_tick,
            velocity=note.velocity,
            channel=note.channel,
            release_velocity=note.release_velocity,
            is_muted=note.is_muted,
            probability=note.probability,
            velocity_deviation=note.velocity_deviation,
        )
        for note in data.notes
    )
    db.commit()
    return to_response(pattern, _notes(db, pattern.pattern_id))
```

문법 풀이

식	뜻
`require_role(member, OWNER, EDITOR)`	둘 중 하나면 통과, 아니면 403 을 던집니다
`db.execute(delete(...).where(...))`	그 패턴의 노트 행을 한 번에 지웁니다
`db.add_all(... for note in data.notes)`	새 노트들을 한꺼번에 넣습니다 (제너레이터로 만듦)
`db.commit()`	지우기와 넣기를 한 커밋으로 확정합니다
`_notes(db, ...)`	커밋 뒤 최신 노트를 다시 읽어 응답에 담습니다

4. 왜 이렇게

선택	이유
통째 교체	피아노롤의 한 편집으로 여러 노트가 바뀝니다(겹침·이동). 노트마다 번호를 주고받으면 어긋나기 쉽습니다. 목록 전체를 다시 보내는 편이 간단하고 결과가 확실합니다
커밋 뒤 다시 읽음	교체 전에 세션이 들고 있던 옛 노트가 응답에 섞이지 않게, 쿼리로 최신 상태를 읽습니다 (214 `_notes`)
빈 목록 허용	화면에서 노트를 다 지운 상태를 그대로 저장할 수 있어야 합니다
5000개 상한	실수로 아주 큰 목록을 보내는 것을 스키마(213)에서 막습니다

5. 시험

`tests/test_pattern_routes.py` 에서 교체 성공, 빈 목록으로 모두 지우기, 비멤버 404, VIEWER 403, 노트 422 를 확인했습니다. 통과합니다.

Backend(FastAPI/Python) 216 — `update_pattern()`

항목	내용
파일	`studionx/backend/app/services/pattern_service.py`
작업	S1-08 아코디언·피아노롤: 패턴 이름·색·길이·스윙 수정 (PRD 02 FR-233)
작성	Claude (AI), 2026-10-17, 사용자 검토
이전 · 다음	215 `replace_notes` · 217 패턴 라우터(목록·상세)

1. 하는 일

패턴의 이름·색·길이·스윙을 고칩니다. **보낸 칸만** 바꾸고, 보내지 않은 칸은 그대로 둡니다. 저장은 OWNER·EDITOR 만 하고(VIEWER 403), 남의 프로젝트 패턴은 404 입니다.

2. 언제 쓰나

쓰는 곳	
`PATCH /patterns/{id}` (218)	패턴 이름 바꾸기, 색 지정, 길이(마디)·스윙 조절
피아노롤 위쪽 도구	패턴 길이(마디)를 바꿀 때 (연결은 S1-10)

3. 코드

```
def update_pattern(
    db: Session, user: User, pattern_id: int, data: PatternUpdateRequest
) -> PatternResponse:
    pattern, _project, member = _find(db, user, pattern_id)
    project_service.require_role(member, MemberRole.OWNER, MemberRole.EDITOR)
    for field, value in data.model_dump(exclude_unset=True).items():
        setattr(pattern, field, value)
    db.commit()
    db.refresh(pattern)
    return to_response(pattern, _notes(db, pattern.pattern_id))
```

문법 풀이

식	뜻
`data.model_dump(exclude_unset=True)`	**보낸 칸만** 꺼냅니다. 안 보낸 칸은 목록에서 빠집니다
`setattr(pattern, field, value)`	모델 행의 그 칸에 값을 넣습니다. `field` 는 칸 이름 문자열입니다
`db.commit()`	바뀐 칸을 확정합니다
`db.refresh(pattern)`	`updated_at` 처럼 DB 가 채우는 칸을 다시 읽습니다

4. 왜 이렇게

선택	이유
`exclude_unset=True`	`PATCH` 는 "보낸 칸만 바꾼다"가 규칙입니다. 안 보낸 칸을 안 건드리려면 이 옵션이 필요합니다
`refresh`	`updated_at` 은 DB 의 `ON UPDATE` 가 채웁니다. 커밋 뒤 다시 읽어야 응답에 최신 값이 담깁니다
이름은 빈 값 금지	스키마(213)의 `_name_not_null` 이 `null` 을 막고, `PatternName` 이 공백만 친 이름을 422 로 막습니다

5. 시험

`tests/test_pattern_routes.py` 에서 이름·색·길이 수정, 보낸 칸만 바뀌는지, VIEWER 403 을 확인했습니다. 통과합니다.

Backend(FastAPI/Python) 217 — 패턴 라우터 `GET /patterns` · `GET /patterns/{id}`

항목	내용
파일	`studionx/backend/app/api/v1/patterns.py`
작업	S1-08 아코디언·피아노롤: 패턴 목록·상세 API (PRD 02 FR-232, FR-233)
작성	Claude (AI), 2026-10-17, 사용자 검토
이전 · 다음	216 `update_pattern` · 218 패턴 라우터(노트·수정)

1. 하는 일

패턴을 읽는 두 길을 엽니다. 라우터는 얇게 두고 로직은 `pattern_service` (214) 로 넘깁니다.

메서드 · 경로	하는 일	권한
`GET /patterns?project_id=`	프로젝트의 패턴 목록 (노트 없이)	멤버면 누구나
`GET /patterns/{id}`	패턴 한 건 + 노트 전부	멤버면 누구나

2. 언제 쓰나

쓰는 곳	
피아노롤	클립을 열 때 패턴 상세로 노트를 받습니다
편곡 화면	프로젝트의 패턴 목록을 받아 클립과 잇습니다 (연결은 S1-10)

3. 코드

```
router = APIRouter()

@router.get("", response_model=PatternListResponse)
def list_patterns(
    user: CurrentUser,
    db: Db,
    project_id: Annotated[int, Query(ge=1)],
) -> PatternListResponse:
    return pattern_service.list_patterns(db, user, project_id)

@router.get(
    "/{pattern_id}",
    response_model=PatternResponse,
    responses={404: {"model": ErrorResponse}},
)
def get_pattern(pattern_id: int, user: CurrentUser, db: Db) -> PatternResponse:
    return pattern_service.get_pattern(db, user, pattern_id)
```

`app/api/v1/router.py` 에 아래 한 줄로 붙입니다.

```
router.include_router(patterns.router, prefix="/patterns", tags=["patterns"])
```

문법 풀이

식	뜻
`CurrentUser` · `Db`	`app/api/deps.py` 의 별칭입니다. 로그인한 사용자와 DB 세션을 자동으로 받습니다
`Annotated[int, Query(ge=1)]`	쿼리 값은 1 이상이어야 합니다 (0·음수는 422)
`response_model=...`	응답을 이 스키마(213) 모양으로 검사해 내보냅니다
`responses={404: {"model": ErrorResponse}}`	문서(OpenAPI)에 404 응답 모양을 적어 둡니다
경로가 `""`	접두사가 `/patterns` 라서 빈 문자열이 목록 경로가 됩니다

4. 왜 이렇게

선택	이유
라우터는 얇게	규칙(권한·정렬·응답)은 서비스 한 곳에 모읍니다. 라우터는 부르기만 합니다
목록은 `project_id` 를 쿼리로	프로젝트의 패턴을 모아 보는 것이 목적이라 프로젝트 번호가 기준입니다
401 은 자동	`CurrentUser` 가 로그인을 요구합니다

5. 시험

`tests/test_pattern_routes.py` 에서 목록(빈 목록·여러 건·정렬), 상세(노트 포함), 401·404 를 확인했습니다. 통과합니다.

Backend(FastAPI/Python) 218 — 패턴 라우터 PUT

`/patterns/{id}/notes` • `PATCH /patterns/{id}`

항목	내용
파일	`studionx/backend/app/api/v1/patterns.py`
작업	S1-08 아코디언·피아노롤: 노트 저장·패턴 수정 API (PRD 02 FR-231, FR-233)
작성	Claude (AI), 2026-10-17, 사용자 검토
이전 · 다음	217 패턴 라우터(목록·상세) · 219 마디 눈금 클릭으로 플레이헤드

1. 하는 일

패턴을 바꾸는 두 길을 엽니다. 둘 다 OWNER·EDITOR 만 되고 VIEWER 는 403 입니다.

메서드 · 경로	하는 일	권한
`PUT /patterns/{id}/notes`	노트를 보낸 목록으로 통째 교체 (215)	OWNER·EDITOR
`PATCH /patterns/{id}`	이름·색·길이·스윙 수정 (216)	OWNER·EDITOR

2. 언제 쓰나

쓰는 곳	
피아노롤	노트를 찍고 고치고 지운 뒤 `PUT …/notes` 로 저장합니다
피아노롤 도구	패턴 이름·길이를 바꿀 때 `PATCH …` 를 씁니다

3. 코드

```
@router.put(
    "/{pattern_id}/notes",
    response_model=PatternResponse,
    responses={403: {"model": ErrorResponse}, 404: {"model": ErrorResponse}},
)
def replace_notes(
    pattern_id: int,
    data: NotesSaveRequest,
    user: CurrentUser,
    db: Db,
) -> PatternResponse:
    return pattern_service.replace_notes(db, user, pattern_id, data)

@router.patch(
    "/{pattern_id}",
    response_model=PatternResponse,
    responses={403: {"model": ErrorResponse}, 404: {"model": ErrorResponse}},
)
def update_pattern(
```

```
    pattern_id: int,
    data: PatternUpdateRequest,
    user: CurrentUser,
    db: Db,
) -> PatternResponse:
    return pattern_service.update_pattern(db, user, pattern_id, data)
```

문법 풀이

식	뜻
`@router.put(...)` · `@router.patch(...)`	각각 통째 교체, 일부 수정에 맞는 메서드입니다
`/{pattern_id}/notes`	그 패턴의 노트 묶음을 가리키는 하위 경로입니다
`data: NotesSaveRequest`	본문(JSON)을 스키마(213)로 검사해 받습니다
`responses={403: ..., 404: ...}`	문서에 두 오류 모양을 적어 둡니다

4. 왜 이렇게

선택	이유
노트는 `PUT`(통째 교체)	여러 노트가 한 번에 바뀌는 편집이라 목록 전체를 보내는 방식이 맞습니다 (215)
패턴 정보는 `PATCH`	이름 하나만 바꾸는 일이 많아 보낸 칸만 바꾸는 방식이 맞습니다 (216)
403·404 를 문서에 명시	권한·없음 규칙을 API 문서에서 바로 알 수 있게 합니다

5. 시험

`tests/test_pattern_routes.py` 에서 노트 교체, 패턴 수정, 403·404·422 를 확인했습니다. 통과합니다.